

Competitive Programming

Section 57 Editorial: Reductions And Modeling

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Matching Model	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Small 2-SAT	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Capstone Matching	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Assignment as Matching. Create a left vertex for each worker and a right vertex for each task. Add an edge when that worker can perform that task. A matching chooses at most one incident edge per worker and per task, exactly matching the assignment constraints. Therefore a matching of size k corresponds to k valid assignments, and a maximum matching gives the maximum number of assignments.

B. 2-SAT as SCC. For every clause $(a \text{ or } b)$, add implications $\text{not } a \rightarrow b$ and $\text{not } b \rightarrow a$. If a variable and its negation are in the same SCC, each implies the other, so both truth values would be forced and the formula is impossible. Otherwise, SCC topological order gives an assignment: choose the later component between x and $\text{not } x$.

C. Modeling Capstone. The capstone asks for the same habit: identify the objects, identify incompatibilities or implications, then choose the graph model whose feasible solutions match the original statement in both directions. The proof must explicitly map original solutions to graph solutions and graph solutions back to original solutions.

Common pitfalls. A reduction is not complete until it is bidirectional. Check that every graph solution satisfies all original constraints, not only that every original solution can be represented.

Each solution starts by building a model graph or implication system.

- A is direct bipartite matching: every selected edge pairs one left item with one right item, and matching enforces uniqueness.
- B builds implications for each 2-SAT clause. The formula is impossible iff a variable and its negation share an SCC.
- C reinforces the same matching reduction on a second instance.

The reduction proof must go both ways: every original solution maps to a model solution, and every model solution maps back to a valid original answer.

2 A. Matching Model

2.1 Restatement

Given a bipartite graph, print maximum matching size.

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

2.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
```

```
match=[-1]*m
def dfs(u,vis):
    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()
```

3 B. Small 2-SAT

3.1 Restatement

Given clauses (a or b) over signed variables, decide satisfiability.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(ci
```

3.6 Full Python solution

```
import sys, itertools
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; clauses=[]; idx=2
    for _ in range(m): clauses.append((data[idx],data[idx+1])); idx+=2
```

```

for mask in range(1<<n):
    ok=True
    for a,b in clauses:
        va=((mask>>(abs(a)-1))&1)==(a>0); vb=((mask>>(abs(b)-1))&1)==(b>0)
        if not (va or vb): ok=False; break
    if ok: print("YES"); return
print("NO")
if __name__=="__main__": main()

```

4 C. Capstone Matching

4.1 Restatement

Another matching instance to reinforce the reduction pattern.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

4.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
```

```

match=[-1]*m
def dfs(u,vis):
    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()

```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.